Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Институт информатики и вычислительной техники

09.03.01 "Информатика и вычислительная техника"
профиль "Программное обеспечение средств
вычислительной техники и автоматизированных систем"

Кафедра прикладной математики и кибернетики

# Курсовая работа по дисциплине

# Алгоритмы и вычислительные методы оптимизации

# Двойственный симплекс-метод

Вариант 3

Выполнил:

студент гр.ИП-111 _____/Гердележов Д.Д./
ФИО студента

«19» Мая 2024 г.

Проверил преподаватель _____/Галкина М.Ю./
ФИО преподавателя

«25» Мая 2024 г. Оценка_____

Новосибирск 2024 г.

# Оглавление

Задание: ..... 3

1. Двойственный симплекс-метод ..... 4

1.1. Теорема двойственности ..... 4

1.2. Алгоритм двойственного симплекс-метода ..... 4

1.3. Пример ..... 5

2. Пример работы программы ..... 8

2.1. Одно решение ..... 8

2.2. Бесконечно много решений ..... 9

2.3. Нет решений ..... 11

3. Список использованных источников ..... 13

4. Листинг ..... 14

fraction.hpp ..... 14

fraction.cpp ..... 14

main.cpp ..... 17

## Задание:

Написать программу, решающую задачу линейного программирования в канонической форме симплекс-методом одним из перечисленных способов:

- симплекс-методом, используя в качестве начальной угловой точки опорное решение, найденное методом Жордана-Гаусса (1);
- методом искусственного базиса (2);
- двойственным симплекс-методом (3).

Номер варианта V выбирается по формуле V = (N mod 3) + 1, где N – номер в таблице.

На вход программе подаются данные задачи линейного программирования, заданной в канонической форме (считываются из файла в виде матрицы размера (m+1)×(n+1)):

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1 \\ \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \\ a_{m1}x_1 + a_{m2}x_2 + \ldots + a_{mn}x_n = b_m \\ x_1, x_2, \ldots, x_n \geq 0 \end{cases}$$
$$Z = c_1x_1 + c_2x_2 + \ldots + c_nx_n \to \max$$

Требования к программе:

- программа работает с классом простых дробей;
- программа находит решение ЗЛП методом по варианту;
- в качестве входных данных подаются матрицы, удовлетворяющие требованиям метода;
- программа должна обрабатывать возможное отсутствие решений;
- если решений бесконечно много, то программа находит конечное число решений (больше одного).

# 1. Двойственный симплекс-метод

Согласно заданию курсовой работы необходимо написать программу, решающую задачу линейного программирования в канонической форме двойственным симплекс-методом.

Двойственный симплекс-метод заключается в построении оптимального недопустимого плана с последующим преобразованием его в допустимый, не нарушая оптимальности.

## 1.1. Теорема двойственности

**Теорема двойственности:** Если из пары двойственных задач одна обладает оптимальным планом, то и другая имеет решение, причем экстремальные значения линейных функций этих задач равны.

## 1.2. Алгоритм двойственного симплекс-метода

1. В столбце свободных членов выбирают среди отрицательных минимальный. Это определяет разрешающую строку.
2. Для отрицательных элементов разрешающей строки находим симплексные отношения: отношения элементов Z-строки к отрицательным элементам разрешающей строки, взятые по модулю.
3. Выбираем минимальное симплексное отношение, соответствующий столбец – разрешающий.
4. Выполняют шаг симплексных преобразований таблицы.
5. Если в столбце свободных членов нет отрицательных, то решение оптимально, иначе переход на п.1.

В случае:

- если в разрешающей строке нет ни одного отрицательного элемента, задача неразрешима
- если в Z-строке найден элемент меньше 0, решение следует продолжить обычным симплекс-методом.

## 1.3. Пример

Решить задачу линейного программирования двойственным симплекс-методом и дать геометрическую интерпретацию процесса поиска оптимального решения.

$$Z = x_1 + x_2 + 2x_3 \to \max$$

$$\begin{cases} x_1 + x_2 + x_3 = 8 \\ x_1 - x_2 \geq 4 \\ x_1 + 2x_2 \geq 6 \\ x_i \geq 0,\ i = 1,2,3 \end{cases}$$

*Решение:* Перейдем к канонической форме.

$$\begin{cases} x_1 + x_2 + x_3 = 8 \\ x_1 - x_2 - x_4 = 4 \\ x_1 + 2x_2 - x_5 = 6 \\ x_i \geq 0,\ i = \overline{1,5} \end{cases}$$

$$\begin{array}{ccccc|c} x_1 & x_2 & x_3 & x_4 & x_5 & \\ \hline 1 & 1 & 1 & 0 & 0 & 8 \\ 1 & -1 & 0 & -1 & 0 & 4 \\ 1 & 2 & 0 & 0 & -1 & 6 \end{array} \begin{array}{l} \\ \times(-1) \\ \times(-1) \end{array} \sim \begin{array}{ccccc|c} 1 & 1 & 1 & 0 & 0 & 8 \\ -1 & 1 & 0 & 1 & 0 & -4 \\ -1 & -2 & 0 & 0 & 1 & -6 \end{array}$$

$x_3, x_4, x_5$ – базисные
$x_1, x_2$ – свободные

Надо избавиться от $x_3$ в функции $Z$: $x_3 = 8 - x_1 - x_2$

$Z = x_1 + x_2 + 2(8 - x_1 - x_2) = 16 - x_1 - x_2 \to \max$

Рисунок 1 – Начало решения двойственным симплекс-методом.

**Таблица 1**

| б.п. | 1 | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ |
|---|---|---|---|---|---|---|
| $x_3$ | 8 | 1 | 1 | 1 | 0 | 0 |
| $x_4$ | -4 | -1 | 1 | 0 | 1 | 0 |
| $x_5$ | -6 | -1 | (-2) | 0 | 0 | 1 ← |
| $Z$ | 16 | 1 | 1 | 0 | 0 | 0 |
| $CO$ | | $\left\lvert\frac{1}{-1}\right\rvert$ | $\left\lvert\frac{1}{-2}\right\rvert$ | – | – | – |

↑

$\min(-4;-6) = -6$, $\min\left(\left\lvert\frac{1}{-1}\right\rvert, \left\lvert\frac{1}{-2}\right\rvert\right) = \frac{1}{2}$

Табл 1 – двойственно допустимая, т.к. в $Z$-строке нет отрицательных элементов, в столбце под 1 есть отрицательное

$X^1 = (0;0;8;-4;-6)$ – псевдоплан

$Z(X^1) = 16$

**Таблица 2**

| б.п. | 1 | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ |
|---|---|---|---|---|---|---|
| $x_3$ | 5 | 1/2 | 0 | 1 | 0 | 1/2 |
| $x_4$ | -7 | (-3/2) | 0 | 0 | 1 | 1/2 ← |
| $x_2$ | 3 | 1/2 | 1 | 0 | 0 | -1/2 |
| $Z$ | 13 | 1/2 | 0 | 0 | 0 | 1/2 |
| $CO$ | | $\frac{1/2}{-3/2}$ | – | – | – | – |

↑

$X^2 = (0;3;5;-7;0)$ – псевдоплан

$Z(X^2) = 13$

Рисунок 2 – Нахождение симплексного отношения и попытка избавиться от отрицательных чисел в результирующем столбце.

**Таблица 2**

| б.п. | 1 | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ |
|---|---|---|---|---|---|---|
| $x_3$ | 5 | 1/2 | 0 | 1 | 0 | 1/2 |
| $x_4$ | -7 | -3/2 | 0 | 0 | 1 | 1/2 |
| $x_2$ | 3 | 1/2 | 1 | 0 | 0 | -1/2 |
| Z | 13 | 1/2 | 0 | 0 | 0 | 1/2 |
| CO | | 1/2 : 3/2 | — | — | — | — |

**Таблица 3**

| б.п. | 1 | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ |
|---|---|---|---|---|---|---|
| $x_3$ | 8/3 | 0 | 0 | 1 | 2 | 2 |
| $x_1$ | 14/3 | 1 | 0 | 0 | -2/3 | -1/3 |
| $x_2$ | 2/3 | 0 | 1 | 0 | 2 | 2 |
| Z | 32/3 | 0 | 0 | 0 | 1/3 | 2/3 |

$5 \to 5 - \frac{1}{2}(-7)\cdot(-\frac{2}{3}) = 5 - \frac{7}{3} = \frac{8}{3}$

$3 \to 3 - (-7)\cdot\frac{1}{2}\cdot(-\frac{2}{3}) = 3 - \frac{7}{3} = \frac{2}{3}$

$13 \to 13 - (-7)\cdot\frac{1}{2}(-\frac{2}{3}) = 13 - \frac{7}{3} = \frac{32}{3}$

$\frac{1}{2} \to \frac{1}{2} - \frac{1}{2}\cdot\frac{1}{2}\cdot(-\frac{2}{3}) = \frac{1}{2} + \frac{1}{6} = \frac{4}{6} = \frac{2}{3}$

$X^3 = (14/3; 2/3; 8/3; 0; 0)$ решение оптим.

$z(X^3) = 32/3 = z_{max}$

Рисунок 3 – Повторение действия и вывод оптимального решения.

Решим задачу графически.

$x_3 = 8 - x_1 - x_2 \geq 0 \Rightarrow x_1 + x_2 \leq 8$

$$\begin{cases} x_1 + x_2 \leq 8 \\ x_1 - x_2 \geq 4 \\ x_1 + 2x_2 \geq 6 \\ x_1, x_2 \geq 0 \end{cases}$$

$z = 16 - x_1 - x_2 \to max$

$grad\, z = (-1; -1)$

(1) $x_1 + x_2 = 8$

| $x_1$ | 0 | 8 |
|---|---|---|
| $x_2$ | 8 | 0 |

(2) $x_1 - x_2 = 4$

| $x_1$ | 0 | 4 |
|---|---|---|
| $x_2$ | -4 | 0 |

(3) $x_1 + 2x_2 = 6$

| $x_1$ | 0 | 6 |
|---|---|---|
| $x_2$ | 3 | 0 |

Рисунок 4 – Строим таблицу координат.

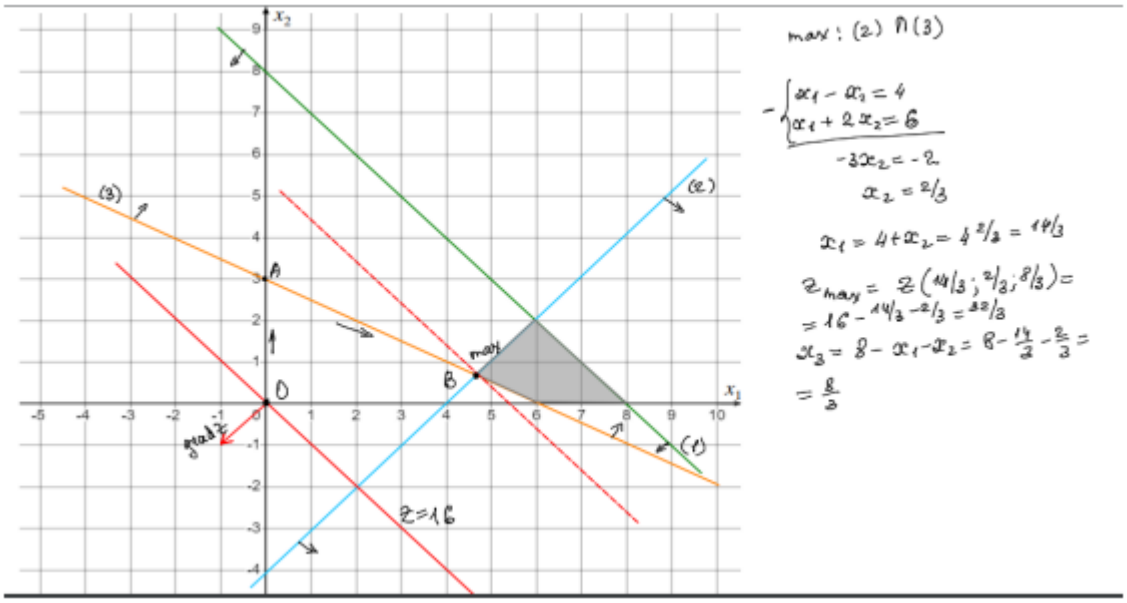



Рисунок 5 – Графическое решение задачи.

Геометрическая интерпретация процесса решения двойственным симплекс-методом.

| Номер таблицы | Решение из симплексной таблицы | Значение функции | Точка на графике |
|---|---|---|---|
| 1 | $X^1 = (8; 0; 8; -4; -6)$ | 16 | 0 |
| 2 | $X^2 = (0; 3; 5; -7; 0)$ | 13 | A |
| 3 | $X^3 = (14/3; 2/3; 8/3; 0; 0)$ | 32/3 | B |

Рисунок 6 – Таблица решений.

# 2. Пример работы программы

## 2.1. Одно решение

Номер 1 из практического занятия №8

```
Условие:
4(x1)+7(x2) >= 57
4(x1)+1(x2) >= 15
3(x1)+11(x2) >= 60
Z = 1(x1)+7(x2) -> min

В каноничной форме:
4(x1)+7(x2)-1(x3) = 57
4(x1)+1(x2)-1(x4) = 15
3(x1)+11(x2)-1(x5) = 60
Z = -1(x1)-7(x2) -> max

До Жордана-Гаусса:
+--------+--------+--------+--------+--------+--------+
|       4|       7|      -1|       0|       0|      57|
+--------+--------+--------+--------+--------+--------+
|       4|       1|       0|      -1|       0|      15|
+--------+--------+--------+--------+--------+--------+
|       3|      11|       0|       0|      -1|      60|
+--------+--------+--------+--------+--------+--------+

После Жордана-Гаусса:
+--------+--------+--------+--------+--------+--------+
|       1|       0|       0|  -11/41|    1/41|  105/41|
+--------+--------+--------+--------+--------+--------+
|       0|       1|       0|    3/41|   -4/41|  195/41|
+--------+--------+--------+--------+--------+--------+
|       0|       0|       1|  -23/41|  -24/41| -552/41|
+--------+--------+--------+--------+--------+--------+

Двойственный симплекс-метод:
+--------+--------+--------+--------+--------+--------+--------+
|   BV   |    1   |   x1   |   x2   |   x3   |   x4   |   x5   |
+--------+--------+--------+--------+--------+--------+--------+
|   x1   |  105/41|       1|       0|       0|  -11/41|    1/41|
+--------+--------+--------+--------+--------+--------+--------+
|   x2   |  195/41|       0|       1|       0|    3/41|   -4/41|
+--------+--------+--------+--------+--------+--------+--------+
|   x3   | -552/41|       0|       0|       1|  -23/41|  -24/41|
+--------+--------+--------+--------+--------+--------+--------+
|    Z   |-1470/41|       0|       0|       0|  -10/41|   27/41|
+--------+--------+--------+--------+--------+--------+--------+
```

```
Z-строка имеет отриц. значения. Новая матрица:
+--------+--------+--------+--------+--------+--------+--------+
|   BV   |    1   |   x1   |   x2   |   x3   |   x4   |   x5   |
+--------+--------+--------+--------+--------+--------+--------+
|   x1   |      20|       1|    11/3|       0|       0|    -1/3|
+--------+--------+--------+--------+--------+--------+--------+
|   x4   |      65|       0|    41/3|       0|       1|    -4/3|
+--------+--------+--------+--------+--------+--------+--------+
|   x3   |      23|       0|    23/3|       1|       0|    -4/3|
+--------+--------+--------+--------+--------+--------+--------+
|    Z   |     -20|       0|    10/3|       0|       0|     1/3|
+--------+--------+--------+--------+--------+--------+--------+


ОТВЕТ:
Z min = -Z max = Z(20; 0) = 20
```

## 2.2. Бесконечно много решений

Номер 2 из практического занятия №5

```
Условие:
1(x1)+1(x2) <= 10
1(x1)-3(x2) <= 3
3(x1)+1(x2) >= 9
-1(x1)+1(x2) <= 4
Z = 1(x1)-1(x2) -> min

В каноничной форме:
1(x1)+1(x2)+1(x3) = 10
1(x1)-3(x2)+1(x4) = 3
3(x1)+1(x2)-1(x5) = 9
-1(x1)+1(x2)+1(x6) = 4
Z = -1(x1)+1(x2) -> max

До Жордана-Гаусса:
+--------+--------+--------+--------+--------+--------+--------+
|       1|       1|       1|       0|       0|       0|      10|
+--------+--------+--------+--------+--------+--------+--------+
|       1|      -3|       0|       1|       0|       0|       3|
+--------+--------+--------+--------+--------+--------+--------+
|       3|       1|       0|       0|      -1|       0|       9|
+--------+--------+--------+--------+--------+--------+--------+
|      -1|       1|       0|       0|       0|       1|       4|
+--------+--------+--------+--------+--------+--------+--------+

После Жордана-Гаусса:
+--------+--------+--------+--------+--------+--------+--------+
|       1|       0|       0|       0|    -1/4|    -1/4|     5/4|
+--------+--------+--------+--------+--------+--------+--------+
|       0|       1|       0|       0|    -1/4|     3/4|    21/4|
+--------+--------+--------+--------+--------+--------+--------+
|       0|       0|       1|       0|     1/2|    -1/2|     7/2|
+--------+--------+--------+--------+--------+--------+--------+
|       0|       0|       0|       1|    -1/2|     5/2|    35/2|
+--------+--------+--------+--------+--------+--------+--------+
```

Двойственный симплекс-метод:

| BV | 1 | x1 | x2 | x3 | x4 | x5 | x6 |
|---|---|---|---|---|---|---|---|
| x1 | 5/4 | 1 | 0 | 0 | 0 | -1/4 | -1/4 |
| x2 | 21/4 | 0 | 1 | 0 | 0 | -1/4 | 3/4 |
| x3 | 7/2 | 0 | 0 | 1 | 0 | 1/2 | -1/2 |
| x4 | 35/2 | 0 | 0 | 0 | 1 | -1/2 | 5/2 |
| Z | 4 | 0 | 0 | 0 | 0 | 0 | 1 |

Решение является оптимальным, но не единственным:

| BV | 1 | x1 | x2 | x3 | x4 | x5 | x6 |
|---|---|---|---|---|---|---|---|
| x1 | 3 | 1 | 0 | 1/2 | 0 | 0 | -1/2 |
| x2 | 7 | 0 | 1 | 1/2 | 0 | 0 | 1/2 |
| x5 | 7 | 0 | 0 | 2 | 0 | 1 | -1 |
| x4 | 21 | 0 | 0 | 1 | 1 | 0 | 2 |
| Z | 4 | 0 | 0 | 0 | 0 | 0 | 1 |

Z(5/4; 21/4) и Z(3; 7)

ОТВЕТ:

Z min = -Z max = Z(5/4-5/4a+3a; 21/4-21/4a+7a) = Z(5/4+7/4a; 21/4+7/4a) = -4  0 <= a <= 1

## 2.3. Нет решений

```
Условие:
1(x1)+1(x2)+2(x3)+1(x4) = 2
2(x1)+1(x2)+3(x3)+1(x4) = 6
Z = 2(x1)+1(x2)-1(x3)-1(x4) -> min

В каноничной форме:
1(x1)+1(x2)+2(x3)+1(x4) = 2
2(x1)+1(x2)+3(x3)+1(x4) = 6
Z = -2(x1)-1(x2)+1(x3)+1(x4) -> max

До Жордана-Гаусса:
+--------+--------+--------+--------+--------+
|       1|       1|       2|       1|       2|
+--------+--------+--------+--------+--------+
|       2|       1|       3|       1|       6|
+--------+--------+--------+--------+--------+

После Жордана-Гаусса:
+--------+--------+--------+--------+--------+
|       1|       0|       1|       0|       4|
+--------+--------+--------+--------+--------+
|       0|       1|       1|       1|      -2|
+--------+--------+--------+--------+--------+

Двойственный симплекс-метод:
+--------+--------+--------+--------+--------+--------+
|   BV   |    1   |   x1   |   x2   |   x3   |   x4   |
+--------+--------+--------+--------+--------+--------+
|   x1   |       4|       1|       0|       1|       0|
+--------+--------+--------+--------+--------+--------+
|   x2   |      -2|       0|       1|       1|       1|
+--------+--------+--------+--------+--------+--------+
|    Z   |      -6|       0|       0|      -4|      -2|
+--------+--------+--------+--------+--------+--------+

Z-строка имеет отриц. значения. Новая матрица:
+--------+--------+--------+--------+--------+--------+
|   BV   |    1   |   x1   |   x2   |   x3   |   x4   |
+--------+--------+--------+--------+--------+--------+
|   x3   |       4|       1|       0|       1|       0|
+--------+--------+--------+--------+--------+--------+
|   x2   |      -6|      -1|       1|       0|       1|
+--------+--------+--------+--------+--------+--------+
|    Z   |      10|       4|       0|       0|      -2|
+--------+--------+--------+--------+--------+--------+
```

Z-строка имеет отриц. значения. Новая матрица:

| BV | 1 | x1 | x2 | x3 | x4 |
|---|---|---|---|---|---|
| x3 | 4 | 1 | 0 | 1 | 0 |
| x4 | -6 | -1 | 1 | 0 | 1 |
| Z | -2 | 2 | 2 | 0 | 0 |

| BV | 1 | x1 | x2 | x3 | x4 |
|---|---|---|---|---|---|
| x3 | 4 | 1 | 0 | 1 | 0 |
| x4 | -6 | -1 | 1 | 0 | 1 |
| Z | -2 | 2 | 2 | 0 | 0 |
| CO | | 2 | - | - | - |

| BV | 1 | x1 | x2 | x3 | x4 |
|---|---|---|---|---|---|
| x3 | -2 | 0 | 1 | 1 | 1 |
| x1 | 6 | 1 | -1 | 0 | -1 |
| Z | -14 | 0 | 4 | 0 | 2 |

Нет решений! [03]

## 3. Список использованных источников


1. Курс лекций Галкиной М.Ю. «Алгоритмы и вычислительные методы оптимизации».
2. Wikipedia *“Симплекс-метод”* [электронный ресурс] // URL: https://ru.wikipedia.org/wiki/%D0%A1%D0%B8%D0%BC%D0%BF%D0%BB%D0%B5%D0%BA%D1%81-%D0%BC%D0%B5%D1%82%D0%BE%D0%B4.
3. Гордеев, С. Н. Интерактивный подход к обучению решения задач двойственным симплекс-методом / С. Н. Гордеев. — Текст : непосредственный // Молодой ученый. — 2017. — № 22 (156). — С. 122-124.
4. Metanit.com *“Руководство по языку программирования C++”* [электронный ресурс] // URL: https://metanit.com/cpp/tutorial/.

# 4. Листинг

## fraction.hpp

```
#include <iostream>

class Fraction {
   public:
    long long numerator, denominator;

    Fraction() : numerator(0), denominator(1) {}
    Fraction(long long n) : numerator(n), denominator(1) {}
    Fraction(long long n, long long d);

    void calculateGcd();
    static Fraction abs(const Fraction& x);

    operator int() { return numerator / denominator; }
    operator float() { return ((float)numerator) / denominator; }
    operator double() { return ((double)numerator) / denominator; }
    Fraction operator+(const Fraction& other) const;
    Fraction& operator+=(const Fraction& other);
    Fraction operator-(const Fraction& other) const;
    Fraction operator-() const;
    Fraction& operator-=(const Fraction& other);
    Fraction operator*(const Fraction& other) const;
    Fraction& operator*=(const Fraction& other);
    Fraction& operator*=(const int& other);
    Fraction operator/(const Fraction& other) const;
    Fraction& operator/=(const Fraction& other);
    Fraction& operator/=(const int& other);
    bool operator==(const Fraction& other) const;
    bool operator<(const Fraction& other) const;
    bool operator>(const Fraction& other) const;

    friend std::ostream& operator<<(std::ostream& os, const Fraction& dt);
    friend std::istream& operator>>(std::istream& is, Fraction& dt);
};
```

## fraction.cpp

```
#include "fraction.hpp"

#include <numeric>
#include <string>

Fraction::Fraction(long long n, long long d) {
    if (d == 0) {
        std::cerr << "Denominator may not be 0." << std::endl;
        exit(0);
```

```
    }
    if (n == 0) {
        numerator = 0;
        denominator = 1;
    } else {
        numerator = n;
        denominator = d;
        calculateGcd();
    }
}

void Fraction::calculateGcd() {
    int sign = 1;
    if (numerator < 0) {
        sign *= -1;
        numerator *= -1;
    }
    if (denominator < 0) {
        sign *= -1;
        denominator *= -1;
    }

    long long tmp = std::gcd(numerator, denominator);
    numerator = numerator / tmp * sign;
    denominator = denominator / tmp;
}

Fraction Fraction::operator+(const Fraction &other) const {
    return Fraction{
        numerator * other.denominator + other.numerator * denominator,
        denominator * other.denominator};
}

Fraction &Fraction::operator+=(const Fraction &other) {
    Fraction tmp =
        Fraction{numerator * other.denominator + other.numerator * denominator,
                 denominator * other.denominator};
    *this = tmp;
    return *this;
}

Fraction Fraction::operator-(const Fraction &other) const {
    return Fraction{
        numerator * other.denominator - other.numerator * denominator,
        denominator * other.denominator};
}

Fraction Fraction::operator-() const {
    return Fraction{-numerator, denominator};
}
```

```
Fraction &Fraction::operator-=(const Fraction &other) {
    Fraction tmp =
        Fraction{numerator * other.denominator - other.numerator * denominator,
                 denominator * other.denominator};
    *this = tmp;
    return *this;
}

Fraction Fraction::operator*(const Fraction &other) const {
    int del = std::gcd(other.numerator, denominator);
    int del2 = std::gcd(numerator, other.denominator);
    return Fraction{(numerator / del2) * (other.numerator / del),
                    (denominator / del) * (other.denominator / del2)};
}

Fraction &Fraction::operator*=(const Fraction &other) {
    int del = std::gcd(other.numerator, denominator);
    int del2 = std::gcd(numerator, other.denominator);
    *this = Fraction{(numerator / del2) * (other.numerator / del),
                     (denominator / del) * (other.denominator / del2)};
    return *this;
}

Fraction &Fraction::operator*=(const int &other) {
    int del = std::gcd(other, denominator);
    Fraction tmp = Fraction{numerator * (other / del), denominator / del};
    *this = tmp;
    return *this;
}

Fraction Fraction::operator/(const Fraction &other) const {
    if (numerator == 0) return Fraction{0};
    int del = std::gcd(other.denominator, denominator);
    int del2 = std::gcd(other.numerator, numerator);
    return Fraction{(numerator / del2) * (other.denominator / del),
                    (denominator / del) * (other.numerator / del2)};
}

Fraction &Fraction::operator/=(const Fraction &other) {
    if (numerator == 0) *this = Fraction{0};
    int del = std::gcd(other.denominator, denominator);
    int del2 = std::gcd(other.numerator, numerator);
    *this = Fraction{(numerator / del2) * (other.denominator / del),
                     (denominator / del) * (other.numerator / del2)};
    return *this;
}

Fraction &Fraction::operator/=(const int &other) {
    int del = std::gcd(other, numerator);
    Fraction tmp = Fraction{numerator / del, denominator * (other / del)};
    *this = tmp;
```

```
    return *this;
}

bool Fraction::operator==(const Fraction &other) const {
    return numerator == other.numerator && denominator == other.denominator;
}

bool Fraction::operator<(const Fraction &other) const {
    return numerator * other.denominator < denominator * other.numerator;
}

bool Fraction::operator>(const Fraction &other) const {
    return numerator * other.denominator > denominator * other.numerator;
}

std::ostream &operator<<(std::ostream &os, const Fraction &dt) {
    std::string number = std::to_string(dt.numerator);
    if (dt.denominator != 1) number += "/" + std::to_string(dt.denominator);
    os << number;
    return os;
}

std::istream &operator>>(std::istream &is, Fraction &dt) {
    std::string str;
    Fraction tmp;
    is >> str;
    int pos = str.find('/');
    if (pos == std::string::npos) {
        dt.numerator = std::stoll(str);
    } else {
        dt.numerator = std::stoll(str.substr(0, pos));
        dt.denominator = std::stoll(str.substr(pos + 1));
        dt.calculateGcd();
    }
    return is;
}

Fraction Fraction::abs(const Fraction &x) {
    Fraction tmp = x;
    if (tmp.numerator < 0) tmp.numerator = -tmp.numerator;
    return tmp;
}
```

## main.cpp

```
#include <algorithm>
#include <fstream>
#include <iomanip>
#include <string>
#include <vector>
```

```
#include "fraction.hpp"

class Matrix {
   private:
    int rows, columns;
    int countOfVariables;
    bool noSolution = false;
    std::vector<std::vector<Fraction>> matrix;
    std::vector<std::string> symbols;
    std::vector<Fraction> Z;
    std::vector<int> basises;
    std::vector<int> free;
    std::vector<Fraction> CO;
    bool checkNegativElements();
    void setMaxElement(int i, int j);
    bool checkZero(int i, int j);
    bool noSolutions();
    void removeZeroVectors();
    bool makeCanon();
    void calcBasisesIndex();
    void GaussStep(const int &row, const int &pos);
    bool SimplexStep();
    std::pair<int, int> negativPos();
    bool checkNegativZ();
    bool checkNegativSolution();
    void SimpexSolution();

   public:
    Matrix(std::ifstream &in);
    bool isMax;
    void printMatrix() const;
    void printStart(const std::string &max) const;
    void printSimplex(const int &row = -1, const int &column = -1);
    bool JordanGauss();
    void Dual_Simplex();
};

Matrix::Matrix(std::ifstream &in) {
    in >> rows >> columns;
    countOfVariables = columns;
    columns++;
    matrix.resize(rows, std::vector<Fraction>(columns));
    symbols.resize(rows);
    Z.resize(columns);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < columns - 1; ++j) {
            in >> matrix[i][j];
        }
        in >> symbols[i];
        in >> matrix[i][columns - 1];
```

```
    }
    for (int i = 0; i < columns - 1; ++i) {
        in >> Z[i];
    }
    Z[columns - 1] = Fraction(0);
    std::string temp;
    in >> temp;
    if (temp == "max")
        isMax = true;
    else
        isMax = false;
}

// PRINT
void Matrix::printMatrix() const {
    int spacing = 9;

    auto printBorder = [](int m_columns, int spacing) -> void {
        for (int i = 0; i < m_columns; ++i) {
            std::cout << "+" << std::string((spacing - 1), '-');
        }
        std::cout << "+" << std::endl;
    };
    printBorder(columns, spacing);
    for (int i = 0; i < rows; ++i) {
        std::cout << "|";
        for (int j = 0; j < columns - 1; ++j) {
            std::cout << std::setw(spacing - 1) << std::right << matrix[i][j]
                      << "|";
        }
        std::cout << "" << std::setw(spacing - 1) << std::right
                  << matrix[i][columns - 1] << "|";
        std::cout << std::endl;
        printBorder(columns, spacing);
    }
    std::cout << std::endl;
}

void Matrix::printStart(const std::string &max) const {
    for (int i = 0; i < rows; ++i) {
        bool isFirst = true;
        for (int j = 0; j < columns - 1; ++j) {
            if (matrix[i][j].numerator != 0) {
                if (matrix[i][j].numerator > -1 && !isFirst) {
                    std::cout << "+";
                }
                std::cout << matrix[i][j] << "(x" << j + 1 << ")";
                isFirst = false;
            }
        }
        std::cout << " " << symbols[i] << " ";
```

```
            std::cout << matrix[i][columns - 1] << std::endl;
        }
        std::cout << "Z = ";
        bool isFirst = true;
        for (int i = 0; i < columns; ++i) {
            if (Z[i].numerator != 0) {
                if (Z[i].numerator > -1 && !isFirst) {
                    std::cout << "+";
                }
                std::cout << Z[i] << "(x" << i + 1 << ")";
                isFirst = false;
            }
        }
        std::cout << " -> " << max << std::endl;
    }

    void Matrix::printSimplex(const int &row, const int &column) {
        int spacing = 9;
        int columnsCt = columns + 1;
        auto printBorder = [](int m_columns, int spacing) -> void {
            for (int i = 0; i < m_columns; ++i) {
                std::cout << "+" << std::string((spacing - 1), '-');
            }
            std::cout << "+" << std::endl;
        };
        printBorder(columnsCt, spacing);
        int half = (spacing - 2) / 2;
        std::cout << "|";
        std::cout << std::string(half, ' ') << "BV"
                  << std::string(spacing - 3 - half, ' ') << "|";
        std::cout << std::string(half + 1, ' ') << "1"
                  << std::string(spacing - 3 - half, ' ') << "|";
        for (int i = 0; i < columns - 1; ++i) {
            std::cout << std::string(half, ' ') << "x" << i + 1 << ""
                      << std::string(spacing - 3 - half, ' ') << "|";
        }
        std::cout << std::endl;
        printBorder(columnsCt, spacing);
        for (int i = 0; i < rows; ++i) {
            std::cout << "|";
            std::cout << std::string(half, ' ') << "x" << basises[i] + 1 << ""
                      << std::string(spacing - 3 - half, ' ') << "|";
            if (row == i) {
                std::cout << "\033[31m" << std::setw(spacing - 1) << std::right
                          << matrix[i][columns - 1] << "\033[0m|";
            } else
                std::cout << "" << std::setw(spacing - 1) << std::right
                          << matrix[i][columns - 1] << "|";
            for (int j = 0; j < columns - 1; ++j) {
                if (i == row && j == column)
                    std::cout << "\033[31m" << std::setw(spacing - 1) << std::right
```

```
                        << matrix[i][j] << "\033[0m|";
            else
                std::cout << std::setw(spacing - 1) << std::right
                        << matrix[i][j] << "|";
        }
        std::cout << std::endl;
        printBorder(columnsCt, spacing);
    }
    std::cout << "|" << std::string(half + 1, ' ') << "Z"
              << std::string(spacing - 3 - half, ' ') << "|";
    std::cout << "" << std::setw(spacing - 1) << std::right << Z[columns - 1]
              << "|";
    for (int i = 0; i < columns - 1; ++i) {
        std::cout << std::setw(spacing - 1) << std::right << Z[i] << "|";
    }
    std::cout << std::endl;
    printBorder(columnsCt, spacing);
    if (row != -1) {
        std::cout << "|" << std::string(half, ' ') << "CO"
                  << std::string(spacing - 3 - half, ' ') << "|";
        std::cout << std::string(spacing - 1, ' ') << "|";
        for (int i = 0; i < columns - 1; ++i) {
            if (CO[i].numerator > 0) {
                if (i == column)
                    std::cout << "\033[31m" << std::setw(spacing - 1)
                            << std::right << CO[i] << "\033[0m|";
                else
                    std::cout << std::setw(spacing - 1) << std::right << CO[i]
                            << "|";
            } else
                std::cout << std::setw(spacing - 1) << std::right << "- "
                        << "|";
        }
        std::cout << std::endl;
        printBorder(columnsCt, spacing);
    }
    std::cout << std::endl;
}

// MATH
bool Matrix::checkNegativElements() {
    bool flag = false;
    for (auto &it : matrix) {
        if (it.back().numerator < 0) {
            flag = true;
            for (auto &i : it) {
                i = -i;
            }
        }
    }
    return flag;
```

```
}

bool Matrix::checkZero(int i, int j) {
    while (i < rows) {
        if (matrix[i][j].numerator != 0) return false;
        ++i;
    }
    return true;
}

void Matrix::setMaxElement(int starti, int j) {
    int swapPos = -1;
    Fraction mmax = Fraction::abs(matrix[starti][j]);
    for (int i = starti + 1; i < rows; ++i) {
        if (Fraction::abs(matrix[i][j]) > mmax) {
            mmax = Fraction::abs(matrix[i][j]);
            swapPos = i;
        }
    }
    if (swapPos != -1) {
        for (int i = 0; i < columns; ++i) {
            std::swap(matrix[starti][i], matrix[swapPos][i]);
        }
    }
}

bool Matrix::noSolutions() {
    for (int i = rows - 1; i > -1; --i) {
        if (matrix[i][columns - 1].numerator != 0) {
            bool isZero = true;
            for (int j = 0; j < columns - 1; ++j) {
                if (matrix[i][j].numerator != 0) {
                    isZero = false;
                    break;
                }
            }
            if (isZero) return true;
        }
    }
    return false;
}

void Matrix::removeZeroVectors() {
    int rowsCt = 0;
    for (auto &it : matrix) {
        if (std::all_of(it.begin(), it.end(), [](const Fraction &val) {
                return val.numerator == 0;
            })) {
            rowsCt++;
        }
    }
```

```
    for (int i = 0; i < rowsCt; ++i) {
        matrix.pop_back();
    }
    rows -= rowsCt;
}

void Matrix::GaussStep(const int &row, const int &pos) {
    Fraction del = matrix[row][pos];
    for (int i = 0; i < columns; ++i) {
        matrix[row][i] /= del;
    }
    for (int i = row + 1; i < rows; ++i) {
        for (int j = pos + 1; j < columns; ++j) {
            matrix[i][j] -= matrix[row][j] * matrix[i][pos];
        }
        for (int j = pos - 1; j > -1; --j) {
            matrix[i][j] -= matrix[row][j] * matrix[i][pos];
        }
        matrix[i][pos].numerator = 0;
        matrix[i][pos].denominator = 1;
    }
    for (int j = pos + 1; j < columns; ++j) {
        Z[j] -= matrix[row][j] * Z[pos];
    }
    for (int j = pos - 1; j > -1; --j) {
        Z[j] -= matrix[row][j] * Z[pos];
    }
    Z[pos].numerator = 0;
    Z[pos].denominator = 1;
    for (int i = row - 1; i > -1; --i) {
        for (int j = pos + 1; j < columns; ++j) {
            matrix[i][j] -= matrix[row][j] * matrix[i][pos];
        }
        for (int j = pos - 1; j > -1; --j) {
            matrix[i][j] -= matrix[row][j] * matrix[i][pos];
        }
        matrix[i][pos].numerator = 0;
        matrix[i][pos].denominator = 1;
    }
}

bool Matrix::JordanGauss() {
    int row = 0;
    int column = 0;

    for (; row < rows && column < columns - 1; ++row, ++column) {
        while (checkZero(row, column) && column < columns - 1) {
            column++;
        }
        if (column == columns - 1) {
            break;
```

```
        }
        setMaxElement(row, column);
        GaussStep(row, column);
    }

    if (noSolutions()) {
        noSolution = true;
    }
    if (rows > columns - 1) noSolution = true;
    removeZeroVectors();
    return !noSolution;
}

bool Matrix::makeCanon() {
    bool flag = false;
    for (int i = 0; i < rows; ++i) {
        if (symbols[i] == "<=") {
            for (int j = 0; j < rows; ++j) {
                matrix[j].push_back(matrix[j].back());
                matrix[j][columns - 1] = Fraction(0);
            }
            Z.push_back(Fraction(0));
            columns++;
            matrix[i][columns - 2] = Fraction(1);
            symbols[i] = "=";
            flag = true;
        } else if (symbols[i] == ">=") {
            for (int j = 0; j < rows; ++j) {
                matrix[j].push_back(matrix[j].back());
                matrix[j][columns - 1] = Fraction(0);
            }
            Z.push_back(Fraction(0));
            columns++;
            matrix[i][columns - 2] = Fraction(-1);
            symbols[i] = "=";
            flag = true;
        }
    }
    if (!isMax) {
        for (int i = 0; i < columns; ++i) {
            Z[i] = -Z[i];
        }
        flag = true;
    }
    return flag;
}

void Matrix::calcBasisesIndex() {
    basises.resize(rows);
    free.resize(columns - 1 - rows);
    for (int j = 0; j < basises.size(); ++j) {
```

```
            basises[j] = j;
        }
        for (int j = 0; j < free.size(); ++j) {
            free[j] = j + basises.size();
        }
    }

    std::pair<int, int> Matrix::negativPos() {
        std::pair<int, int> pos = {-1, -1};
        Fraction mmin = Fraction(0);
        for (int i = 0; i < rows; ++i) {
            if (matrix[i].back().numerator < mmin.numerator) {
                bool flag = false;
                for (int j = 0; j < columns - 1; j++) {
                    if (matrix[i][j].numerator < 0) {
                        flag = true;
                        break;
                    }
                }
                if (flag) {
                    mmin = matrix[i].back();
                    pos.first = i;
                }
            }
        }
        if (pos.first == -1) return pos;
        mmin = Fraction(-1);
        for (int i = 0; i < columns - 1; ++i) {
            if (matrix[pos.first][i].numerator < 0 && Z[i].numerator >= 0) {
                CO[i] = Fraction::abs(Z[i] / matrix[pos.first][i]);
                if (mmin.numerator == -1 || mmin > CO[i]) {
                    mmin = CO[i];
                    pos.second = i;
                }
            } else {
                CO[i] = Fraction(-1);
            }
        }
        return pos;
    }

    bool Matrix::checkNegativZ() {
        bool flag = true;
        for (int i = 0; i < columns - 1; ++i) {
            if (Z[i].numerator < 0) {
                flag = false;
                Fraction mmax = Fraction(999999999);
                int row = -1;
                for (int j = 0; j < rows; ++j) {
                    if (matrix[j][i] < mmax && matrix[j][i].numerator > 0) {
                        mmax = matrix[j][i];
```

```
                    row = j;
                    flag = true;
                }
            }
            if (flag) {
                GaussStep(row, i);
                free.erase(std::find(free.begin(), free.end(), i));
                free.push_back(basises[row]);
                basises[row] = i;
                i = -1;
                std::cout << "Z-строка имеет отриц. значения. Новая матрица:"
                          << std::endl;
                printSimplex();
            } else
                break;
        }
    }
    return flag;
}

bool Matrix::checkNegativSolution() {
    for (auto &it : matrix) {
        if (it.back().numerator < 0) {
            return false;
        }
    }
    return true;
}

bool Matrix::SimplexStep() {
    std::pair<int, int> pos = negativPos();
    if (pos.second == -1) return false;
    printSimplex(pos.first, pos.second);
    GaussStep(pos.first, pos.second);
    free.erase(std::find(free.begin(), free.end(), pos.second));
    free.push_back(basises[pos.first]);
    basises[pos.first] = pos.second;
    printSimplex();
    return true;
}

void Matrix::SimpexSolution() {
    std::vector<Fraction> solution(countOfVariables, 0);
    for (int i = 0; i < rows; ++i) {
        if (basises[i] >= 0 && basises[i] < solution.size()) {
            solution[basises[i]] = matrix[i].back();
        }
    }
    int hasMoreSolutonsPos = -1;
    for (auto &it : free) {
        if (it >= 0 && it < Z.size() && Z[it].numerator == 0) {
```

```
            for (int i = 0; i < rows; ++i) {
                if (matrix[i][it].numerator > 0) {
                    hasMoreSolutonsPos = it;
                    break;
                }
            }
        }
    }
    if (hasMoreSolutonsPos > -1) {
        std::cout << "Решение является оптимальным, но не единственным:"
                  << std::endl;
        Fraction mmax = Fraction(-1);
        int pos = -1;
        for (int i = 0; i < rows; ++i) {
            if (matrix[i][hasMoreSolutonsPos].numerator > 0) {
                CO[i] = matrix[i].back() / matrix[i][hasMoreSolutonsPos];
                if (mmax.numerator == -1 || mmax > CO[i]) {
                    mmax = CO[i];
                    pos = i;
                }
            } else {
                CO[i] = Fraction(-1);
            }
        }
        GaussStep(pos, hasMoreSolutonsPos);
        free.erase(std::find(free.begin(), free.end(), hasMoreSolutonsPos));
        free.push_back(basises[pos]);
        basises[pos] = hasMoreSolutonsPos;
        printSimplex();
        std::vector<Fraction> solution2(countOfVariables, 0);
        for (int i = 0; i < rows; ++i) {
            if (basises[i] >= 0 && basises[i] < solution2.size()) {
                solution2[basises[i]] = matrix[i].back();
            }
        }
        std::cout << "Z(" << solution[0];
        for (int i = 1; i < solution.size(); ++i) {
            std::cout << "; " << solution[i];
        }
        std::cout << ") и Z(" << solution2[0];
        for (int i = 1; i < solution2.size(); ++i) {
            std::cout << "; " << solution2[i];
        }
        std::cout << ")" << std::endl;
        std::cout << "\nОТВЕТ:\n" << std::endl;
        if (isMax) {
            std::cout << "Z max = Z(";
        } else {
            Z.back() = -Z.back();
            std::cout << "Z min = -Z max = Z(";
        }
```

```
            std::cout << solution[0] << (solution[0].numerator >= 0 ? "-" : "+")
                      << Fraction::abs(solution[0]) << "a"
                      << (solution2[0].numerator >= 0 ? "+" : "-")
                      << Fraction::abs(solution2[0]) << "a";
            for (int i = 1; i < solution.size(); ++i) {
                std::cout << "; " << solution[i]
                          << (solution[i].numerator >= 0 ? "-" : "+")
                          << Fraction::abs(solution[i]) << "a"
                          << (solution2[i].numerator >= 0 ? "+" : "-")
                          << Fraction::abs(solution2[i]) << "a";
            }
            Fraction temp = solution2[0] - solution[0];
            std::cout << ") = Z(" << solution[0] << (temp.numerator >= 0 ? "+" : "")
                      << temp << "a";
            for (int i = 1; i < solution.size(); ++i) {
                temp = solution2[i] - solution[i];
                std::cout << "; " << solution[i] << (temp.numerator >= 0 ? "+" : "")
                          << temp << "a";
            }
            std::cout << ") = " << Z.back() << "\t 0 <= a <= 1" << std::endl;
        } else {
            std::cout << "\nОТВЕТ:" << std::endl;
            if (isMax) {
                std::cout << "Z max = Z(" << solution[0];
            } else {
                Z.back() = -Z.back();
                std::cout << "Z min = -Z max = Z(" << solution[0];
            }
            for (int i = 1; i < solution.size(); ++i) {
                std::cout << "; " << solution[i];
            }
            std::cout << ") = " << Z.back() << std::endl;
        }
    }

    void Matrix::Dual_Simplex() {
        std::cout << std::endl << "Условие:" << std::endl;
        printStart(isMax ? "max" : "min");
        bool flag = true;

        for (int i = 0; i < columns; ++i) {
            if (Z[i].numerator != 0) {
                flag = false;
                break;
            }
        }
        if (flag) {
            std::cout << "Z " << (isMax ? "max" : "min") << " = 0" << std ::endl;
            return;
        }
```

```
    std::cout << std::endl;
    if (makeCanon()) {
        std::cout << "В каноничной форме:" << std::endl;
        printStart("max");
        std::cout << std::endl;
    }

    std::cout << "До Жордана-Гаусса:" << std::endl;
    printMatrix();
    if (checkNegativElements()) {
        std::cout << "Все значения с правой стороны изменены на неотрицательные"
                  << std::endl;
        printMatrix();
    }
    for (int i = 0; i < columns - 1; ++i) {
        Z[i] = -Z[i];
    }
    bool isGauss = JordanGauss();

    std::cout << "После Жордана-Гаусса:" << std::endl;
    printMatrix();
    if (noSolutions() || !isGauss) {
        std::cout << "\nНет решений! [01]\n" << std::endl;
        noSolution = true;
        return;
    }

    calcBasisesIndex();
    CO.resize(columns - 1, Fraction(-1));
    std::cout << "Двойственный симплекс-метод:" << std::endl;
    printSimplex();
    if (!checkNegativZ()) {
        std::cout << "\nНет решений! [02]\n" << std::endl;
        noSolution = true;
        return;
    }
    while (SimplexStep());
    if (!checkNegativSolution()) {
        std::cout << "\nНет решений! [03]\n" << std::endl;
        noSolution = true;
        return;
    }
    SimpexSolution();
}

int main(int argc, char **argv) {
    if (argc < 2) {
        std::cerr << "./main <filename>" << std::endl;
        exit(EXIT_FAILURE);
    }
    std::ifstream in(argv[1]);
```

```
    Matrix matrix(in);
    matrix.Dual_Simplex();
    return 0;
}
```